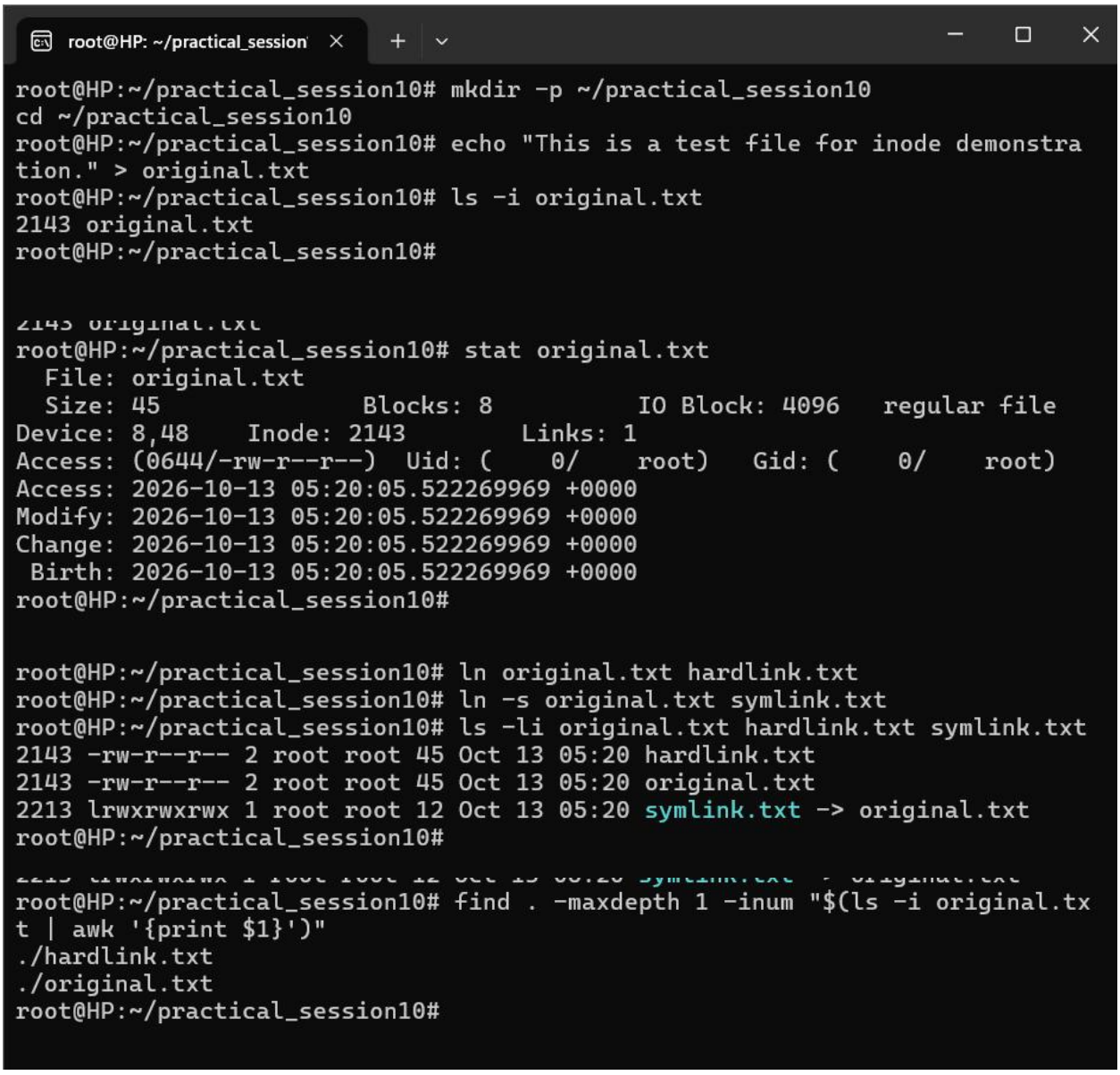
P NEEHARIKA-2520090233

PRACTICAL SESSION 10

Task 1 :- Investigate inode structures using commands such as `ls -i`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.



Output Description :- The output displays inode information for the original file, hard link, and symbolic link. The `find` command identifies the original file and hard link sharing the same inode.

Task 2 :- Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
  GNU nano 7.2                          task2_mmap.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>

#define SIZE 4096

int main() {
    int src, dest;
    char buffer[SIZE];
    ssize_t n;
    struct stat st;
    clock_t start, end;
    double read_write_time, mmap_time;

    FILE *fp = fopen("source.txt", "w");

    if (fp == NULL) {
        perror("fopen");
        return 1;
    }

    for (int i = 0; i < 1000; i++)
        fprintf(fp, "Sample data for mmap demonstration.\n");

    fclose(fp);

    src = open("source.txt", O_RDONLY);

    if (src == -1 || fstat(src, &st) == -1) {
        perror("open/fstat");
        if (src != -1)
            close(src);
```

```
 GNU nano 7.2                          task2_mmap.c *
        return 1;
    }

    size_t size = st.st_size;

    if (size == 0) {
        printf("Source file is empty.\n");
        close(src);
        return 1;
    }

    /* Traditional read/write copy */
    dest = open("readwrite.txt",
                O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest == -1) {
        perror("destination");
        close(src);
        return 1;
    }

    start = clock();

    while ((n = read(src, buffer, SIZE)) > 0) {
        ssize_t total = 0;

        while (total < n) {
            ssize_t w = write(dest, buffer + total, n - total);

            if (w <= 0) {
                perror("write");
                close(src);
                close(dest);
                return 1;
            }

            total += w;
```

```
        }
    }

    end = clock();
    read_write_time = (double)(end - start) / CLOCKS_PER_SEC;

    close(src);
    close(dest);

    /* Memory-mapped file copy */
    src = open("source.txt", O_RDONLY);
    dest = open("mmap_copy.txt",
                O_RDWR | O_CREAT | O_TRUNC, 0644);

    if (src == -1 || dest == -1) {
        perror("open");
        if (src != -1) close(src);
        if (dest != -1) close(dest);
        return 1;
    }

    if (ftruncate(dest, size) == -1) {
        perror("ftruncate");
        close(src);
        close(dest);
        return 1;
    }

    start = clock();

    char *source_map = mmap(NULL, size, PROT_READ,
                            MAP_PRIVATE, src, 0);

    char *dest_map = mmap(NULL, size, PROT_READ | PROT_WRITE,
                          MAP_SHARED, dest, 0);

    if (source_map == MAP_FAILED || dest_map == MAP_FAILED) {
```

```
        perror("mmap");
        if (source_map != MAP_FAILED)
            munmap(source_map, size);
        if (dest_map != MAP_FAILED)
            munmap(dest_map, size);
        close(src);
        close(dest);
        return 1;
    }

    memcpy(dest_map, source_map, size);
    msync(dest_map, size, MS_SYNC);

    end = clock();
    mmap_time = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Memory-mapped I/O Demonstration\n");
    printf("File size: %zu bytes\n", size);
    printf("Traditional read/write copy completed.\n");
    printf("mmap() copy completed successfully.\n");

    printf("\nPerformance Comparison:\n");
    printf("read/write time: %.6f seconds\n", read_write_time);
    printf("mmap() time:     %.6f seconds\n", mmap_time);

    printf("\nData read using mmap():\n");
    printf("%.70s\n", dest_map);

    munmap(source_map, size);
    munmap(dest_map, size);
    close(src);
    close(dest);

    printf("\nMemory mappings released successfully.\n");

    return 0;
}
```

Output Description :- The output displays successful file copying using traditional `read()`/`write()` and `mmap()`, along with their execution times. The copied files are verified to contain the same
data.

```
root@HP:~/practical_session10# nano task2_mmap.c
root@HP:~/practical_session10# gcc task2_mmap.c -o task2_mmap
root@HP:~/practical_session10# ./task2_mmap
Memory-mapped I/O Demonstration
File size: 36000 bytes
Traditional read/write copy completed.
mmap() copy completed successfully.

Performance Comparison:
read/write time: 0.000041 seconds
mmap() time:     0.000237 seconds

Data read using mmap():
Sample data for mmap demonstration.
Sample data for mmap demonstration

Memory mappings released successfully.
root@HP:~/practical_session10#
```

```
root@HP:~/practical_session10# head -n 3 readwrite.txt
head -n 3 mmap_copy.txt
Sample data for mmap demonstration.
Sample data for mmap demonstration.
Sample data for mmap demonstration.
Sample data for mmap demonstration.
Sample data for mmap demonstration.
Sample data for mmap demonstration.
root@HP:~/practical_session10#
```

Observation :- Inode structures, hard links, symbolic links, and memory-mapped file I/O were successfully demonstrated and tested.